

VERIQO Backlog Closure Report

Round: Constitutional Integration & Proof Audit **Branch:** `c:laude/199-gap-coverage-nv70zy` **Date:** 2026-09-03 **Source:** *Backlogs yang harus dibuat atau ditutup*

0. Read this first

Two things in this report will look like bad news and are not.

Nothing reaches QUALIFIED. The constitutional proof audit ends with 0 of 30 articles QUALIFIED. That is not a failure of this round's work; it is the round's main finding. QUALIFIED requires an outside party to have examined the control, and no outside party has examined any of them. Reporting that plainly is the whole point of building the matrix.

Two of the "three internal gaps" were already closed. The backlog named payment lifecycle and audit unification as still PARTIAL. They were closed in Round 9, and this round verified that executably rather than trusting the row. The correction is reported here rather than quietly absorbed.

1. Item-by-item closure

#	Backlog item	Verdict	Where
1	Constitutional Integration & Proof Audit — the matrix and five verdicts	CLOSED	<code>pkg/assurance</code> (<code>matrix.go</code> , 30 articles)
2	VERIQO Proof Object — 23 components, cryptographically bound	CLOSED	<code>pkg/proof</code>
3	Timestamp distinction — chain $\neq$ RFC 3161 attestation	CLOSED	<code>pkg/platform/timestamp</code> , ADR-0003
4	Engineering wave vs Assurance qualification — separate axes	CLOSED	<code>pkg/assurance</code> , ADR-0004
5a	Internal gap: payment lifecycle	VERIFIED CLOSED (was closed in Round 9)	<code>test/integration/internal_gaps_test.go</code>
5b	Internal gap: audit subsystem unification	VERIFIED CLOSED + extended	<code>pkg/casefabric/audit.go</code>
5c	Internal gap: arbitration → dispute evidence support	REFRAMED + ENFORCED	<code>docs/architecture/DISPUTE_EVIDENCE_SUPPORT.md</code> , ADR-0005
6	Case Resolution Fabric — one canonical case for all domains	CLOSED	<code>pkg/casefabric</code>
7	Forward–Reverse as an explicit architecture contract	CLOSED	<code>pkg/fref</code> , <code>docs/architecture/FORWARD_REVERSE_ARCHITECTURE_CONTRACT.md</code>
8	Canonical vocabulary freeze — five fabrics	CLOSED	<code>docs/architecture/VERIQO_CANONICAL_VOCABULARY.md</code> , ADR-0001
9	Five-fabric capability audit (11 dimensions each)	CLOSED	<code>pkg/assurance/fabric.go</code> , <code>docs/architecture/FIVE_FABRIC_CAPABILITY_AUDIT.md</code>
—	<i>Deep search: missing canonical objects</i>	CLOSED — 34 → 40	<code>pkg/ontology</code>
—	<i>Enterprise-grade repo structure</i>	CLOSED	<code>test/architecture/layering_test.go</code> , <code>docs/adr/</code> , <code>docs/architecture/REPOSITORY_STRUCTURE.md</code>

2. Item 1 — the Constitutional Integration & Proof Audit

The chain the backlog specified is executable, not a spreadsheet:

CONSTITUTION → CONTROL → CODE → TEST → EVIDENCE → REPLAY → QUALIFICATION → EXTERNAL PROOF

with the five verdicts applied in the order the backlog states them, most fundamental failure first:

Verdict	Means	Count
OPEN	the article reaches no runtime enforcement	3
INTEGRATION_GAP	code exists, nothing calls it	0
ASSURANCE_GAP	tests exist, no production-path evidence	18
EXTERNAL_QUALIFICATION	internally proved; needs vendor/data/infrastructure	9
QUALIFIED	complete end to end	0

Three design decisions make the matrix worth reading:

- Open is the zero Verdict.** An article nobody traced is open, never qualified. A matrix that silently skipped unfilled rows is the failure mode this exercise exists to prevent, so `Assemble` emits a row for every article whether or not a trace exists.
- A link asserted with no reference is refused.** `TestAssertedLinkNeedsAreference` covers all seven links. This is what keeps a traceability matrix from becoming a spreadsheet of good intentions.
- Impossible chains are refused.** Claiming production evidence for a control nothing calls is a contradiction, not a strong result, and `Assess` returns an error rather than a verdict.

The three `OPEN` articles are named and the count is asserted by test (`TestArticlesWithNoRuntimeEnforcementAreOpen`), so it cannot drift in either direction:

- **Article 9** (ZKP bounded meaning) — no prover or verifier exists.
- **Article 15** (no outcome-contingent neutrality) — a commercial-structure commitment, verifiable only by examining VERIQO's contracts and remuneration.
- **Article 18** (no visual-only redaction) — the PDF/XLSX/PPTX redaction workers and the adversarial recovery lab that would prove byte-level absence are not built.

3. Item 2 — the VERIQO Proof Object

All twenty-three components, sealed with a JCS canonical hash. The pipeline the backlog drew is the package's type system:

PROPOSITION	→	PROOF	OBJECT	→	SUPPORT		CONTRADICT		UNKNOWN	→	QUALIFICATION
	→	SUFFICIENT		→	FINDING	→	AUTHORIZED	→	DECISION		
	→	INSUFFICIENT		→	NEXT	BEST	EVIDENCE				

Stance and sufficiency are derived, never asserted. `Seal` overwrites whatever the author wrote. A conclusion cannot be made sufficient by writing the word — `TestAuthorDeclaredSufficiencyIsOverridden`.

Skipping a stage does not compile. `Decision` can be built only from an `AuthorizedFinding`, which only from a `Finding`, which only from a sealed, sufficient object whose hash re-verifies. Every stage type has unexported fields and every accessor returns a copy.

The authorizer may not be the author. A pipeline that adopts its own conclusions has no authority boundary at all.

Absence is load-bearing. `Unknown` is the zero `Stance`, `NotDetermined` the zero `Sufficiency`, `NotSought` the zero `ExternalStatus`. An object whose author forgot says *we do not know*.

Sufficiency is conjunctive and fails closed. A `SUPPORT` stance is necessary and nowhere near enough: an unresolved material contradiction, an unassessed source set, unassessed quality, an unreviewed material AI contribution, or an incomplete reverse proof each defeat it — and `InsufficiencyReason` says which.

4. Item 3 — the timestamp distinction

VERIQO self-hosted timestamp	=	tamper-evident temporal chain
External TSA	=	independent temporal attestation

`Kind.ProvesExistenceBefore()` returns true only for `IndependentAttestation`. `Attestation.Kind` is **derived by `Assess`**, never settable — the field that decides whether "this existed before then" may be claimed is not one anybody can write.

The adversarial case the backlog implies: **a TSA VERIQO operates itself**. It does not raise the attestation to independent. The token is retained, the downgrade is explained, and `Describe` discloses that a token is present but was not counted — including when there is no chain at all, which was a defect this round's adversarial suite found and fixed (§9).

`ChainAttestation` deliberately carries **no wall-clock field**. Adding one would invite reading it as an attested time.

5. Item 4 — two axes that never combine

ENGINEERING	NOT_STARTED	→	DESIGNED	→	SCAFFOLDED	→	IMPLEMENTED	→	UNIT_TESTED
		→	INTEGRATION_TESTED	→	ADVERSARIAL_TESTED	→	REPLAY_VERIFIED		
ASSURANCE	UNPROVED	→	SELF_ASSERTED	→	INTERNALLY_PROVED				
		→	EXTERNALLY_VALIDATED	→	PRODUCTION_QUALIFIED				

`Status` has no `Overall()`, no `Score()`, no `Percent()`. `AxisReport` is asserted by test to contain no `%` character. Everything from `EXTERNALLY_VALIDATED` up satisfies `RequiresOutsideParty()`, so no amount of further engineering reaches it.

8 of 15 capabilities are adversarially tested or better. 0 of 15 have been examined by anyone outside VERIQO. That shape — not any single figure — is VERIQO's actual position.

A status below `EXTERNALLY_VALIDATED` that names no blocker is refused, because "we have not got round to it" and "no accredited body exists for this" are different situations.

6. Item 5 — the three internal gaps

5a · Payment lifecycle — VERIFIED CLOSED

The backlog read a status from an earlier report. `INSURANCE_COMPLETENESS_AUDIT.json` moved Payment `PARTIAL` → `IMPLEMENTED` in Round 9. Rather than trust the row, this round walks the lifecycle: `authorize` → `instruct` → `settle`, with **segregation of duties enforced** (`RoleInsurer` authorizes; only `RoleBankTradeFinance` may instruct — a disjoint role set, not the same check twice).

The honest limit is asserted too: `ReconcileSettlement` refuses a payment with no external settlement evidence. VERIQO can model the lifecycle end to end; the confirmation that money moved comes from a bank, and nothing in the repository can manufacture it. Payment sits at `IMPLEMENTED` / `SELF_ASSERTED`.

5b · Audit unification — VERIFIED CLOSED, and extended

The gap was never two ledgers. `pkg/platform/audit.AuditStore.Append` has always been the single append primitive; `auditlink`, `decision` and `action` all mirror into it.

The **real remaining gap** was that the mirror was domain-specific: a canonical case, which by construction is not insurance's, had nowhere to go. Closed by `pkg/casefabric/audit.go`, which mirrors a case's timeline into the same store through the same canonical envelope, producing both audit records and an `event.Chain` — two views of one history, not two histories.

Two controls at that boundary:

- **An unverifiable timeline is not mirrored.** Feeding a history that does not verify into the one ledger everything trusts would contaminate it.
- **MirrorProof carries verdicts, not evidence.** Proof hash, stance, sufficiency, qualification, evidence *count* — never statements or content hashes. Copying evidence into an audit log is how disclosure controls get bypassed.

5c · Arbitration — REFRAMED

The Arbitration – PARTIAL row measured VERIQO against a capability it must never have. Building it would not close a gap; it would open the largest one in the system, against Article 16.

The row is now Dispute Evidence Support – IMPLEMENTED, and the completeness audit reads **29/29**. The boundary is enforced in code at four layers: proof.Decide, casefabric.Outcome.Validate, the qualification state set (which has no PROVEN or LIABLE value), and dispute.Position (which carries no field that could mark a contention correct).

This is not modesty. An evidence system that also decides has an interest in the outcome, and an interested party's evidence is worth less to a tribunal. The refusal is what makes the output usable.

7. Items 6–9 — the fabrics

Case Resolution Fabric (pkg/casefabric)

One case spine, nine canonical phases, **six domains projected onto it** — insurance, maritime, commodity, supply chain, trade finance, dispute. A domain keeps its richer vocabulary; every one of its states must map to a canonical phase, and an unmapped state is refused. TestEveryInsuranceStateMapsOntoTheFabric fails the build if a state is added to casestate without a mapping — Law 3 enforced, not asserted.

A case cannot resolve past an unproven material claim or an untested rival hypothesis. The established *and unestablished* claim lists are computed by the fabric rather than accepted from the caller, so a resolution cannot quietly omit what it failed to establish.

Forward–Reverse Execution Fabric (pkg/fref)

Fourteen stages, each bound to the package that owns it and each stating what it **refuses**. Ordering is enforced: TRUST precedes FINDING, and a reverse run that stops at QUALIFICATION is *incomplete* — "we know what to get next" is the successful end of a reverse run.

Closure is the check that the two directions are one architecture: it fails when the finding rests on evidence no proof obligation called for, or when an obligation was never met.

Vocabulary freeze

Fifteen ambiguous terms retired to five fabrics, each to exactly one. TestNoFabricHasItsOwnPackage fails the build if pkg/tecp, pkg/eqf, pkg/crf, pkg/intelligentfabric, pkg/trustengine or pkg/caseengine appears — the five are capabilities over existing code, not five new façades.

Five-fabric capability audit

All five audited on all eleven dimensions, with every canonical package path resolved against the filesystem by test. Each fabric must **name the packages that look like a rival authority** and say why each is not one — the rows are written to be contested, not believed.

8. Deep search — what was missing

Canonical objects: 34 → 40. Six types were values buried inside other objects, which is how a thing stops being addressable — you cannot link to it, version it, or ask what depends on it:

Proposition · ProofObject · Qualification · Contradiction · ProofObligation · NextBestEvidence · Attestation

(Seven names, forty types: Attestation completes the set the architecture states.)

Enterprise-grade structure. Not cosmetics — six layering rules enforced against the real import graph from go list:

Rule	Prevents
Dependencies flow downwards	the general case
Foundation imports only foundation	a bottom-layer package reaching into an unclassified corner and passing the general rule
The constitution depends on no implementation	an article being weakened to match what the code happens to do
Assurance is imported by nothing	the audit becoming part of what it audits
No fabric imports a domain	Law 1 — one canonical state, many projections
Every governed layer has packages	a rule that passes by governing nothing

Plus five ADRs recording the decisions and what each costs.

9. What the adversarial suite found

Nineteen new adversarial cases, written as *attempts* rather than checks. Two found real defects in this round's own code, both fixed:

1. **casefabric.Outcome.Validate split words on underscore**, so liable_party and prevailing_party broke into halves matching nothing. A summary reading "respondent found liable_party" passed. Fixed by treating _ as a word character.
2. **timestamp.Describe hid an uncounted token** when no chain was present: it said "No temporal attestation" without disclosing that a self-run TSA's token existed. A reader who found the token later would reasonably assume it was overlooked. Fixed to disclose it in that branch too.

Both were found by tests written to attack, not to confirm. That is the argument for writing them that way.

10. Verification

scripts/verify.sh, all six stages: build, vet, gofmt, the zero-external-dependency invariant, the full suite under -race, and race-repeat 5× on consensus-critical packages.

Package	New tests	Coverage
pkg/proof	51	80.9%
pkg/assurance	46	88.4%
pkg/casefabric	35	79.9%
pkg/fref	26	94.7%
pkg/platform/timestamp	17	91.7%
test/adversarial (new)	19	—
test/integration (new)	19	—
test/architecture	6	—
Total new	219	

TestFiveFabricsAreOneSystem is the end-to-end proof: one proposition carried through TECP evidence pinning and temporal attestation, an EQF qualification with a gated absence, a sealed Proof Object, a CRF case, both FREF directions, a closure check, the canonical ledger, and five independent re-verifications — ending in a decision that traces back to the evidence it rests on, with the temporal limitation surviving into the case outcome.

It is one long test on purpose. Split into five it would prove each fabric works and prove nothing about whether they compose, which is the only question it exists to answer.

11. Claims this round does not support

1. That anything is QUALIFIED. Nothing is. No outside party has examined any control.
2. That the five fabrics are correct, only that they compose. Every end-to-end test is VERIQO's own.
3. That any evidence in the system carries an independent temporal attestation. VERIQO holds no TSA relationship.
4. That the dispute evidence model matches how disputes are actually pleaded. No counsel has confirmed it; no tribunal has been offered a package.
5. That payment settlement has ever been reconciled against a real bank confirmation.
6. That the layering rules govern the whole repository. Unclassified packages are genuinely exempt, and there are many.
7. That the duplication risks named in the fabric audit are the only ones. They are the ones this audit found, and the rows are written to be contested.

12. Status per MIP §34

Constitutional proof audit	IMPLEMENTED · UNIT_TESTED · ADVERSARIAL_TESTED
Proof Object + pipeline	IMPLEMENTED · UNIT_TESTED · ADVERSARIAL_TESTED
Temporal attestation distinction ..	IMPLEMENTED · UNIT_TESTED · ADVERSARIAL_TESTED
Two-axis assurance model	IMPLEMENTED · UNIT_TESTED · ADVERSARIAL_TESTED
Case Resolution Fabric	IMPLEMENTED · UNIT_TESTED · ADVERSARIAL_TESTED
Forward-Reverse contract	IMPLEMENTED · UNIT_TESTED · ADVERSARIAL_TESTED
Five-fabric capability audit	IMPLEMENTED · UNIT_TESTED
Vocabulary freeze	IMPLEMENTED (enforced by test)
Repository layering	IMPLEMENTED · UNIT_TESTED
Canonical objects (40)	IMPLEMENTED
Dispute evidence support	IMPLEMENTED · ADVERSARIAL_TESTED
Payment lifecycle	IMPLEMENTED · INTEGRATION_TESTED (settlement SELF_ASSERTED)
Audit unification	IMPLEMENTED · INTEGRATION_TESTED
Everything external	READY_FOR_EXTERNAL_QUALIFICATION

No line reads DONE.

Appendix A — the matrix and the two axes, as generated

ART	VERDICT	CONTROL
1	EXTERNAL_QUALIFICATION	Finding requires a sealed proof object with a pinned evidence set
2	ASSURANCE_GAP	Acquisition records provenance without conferring qualification
3	ASSURANCE_GAP	Transitive source clustering: same-root data counts once
4	ASSURANCE_GAP	Rights are evaluated before contact, not after
5	EXTERNAL_QUALIFICATION	Raw bytes are preserved before any transformation
6	EXTERNAL_QUALIFICATION	A finalized version is structurally unupdatable
7	ASSURANCE_GAP	Historical cases resolve against their historical policy version
8	EXTERNAL_QUALIFICATION	AI cannot create, alter, qualify or sign evidence
9	OPEN	A zero-knowledge proof establishes only its stated predicate
10	EXTERNAL_QUALIFICATION	Replay is verifiable without trusting the runtime
11	ASSURANCE_GAP	Dissent is carried through qualification, never deleted
12	ASSURANCE_GAP	The same policy applies to every party absent an authorized exc...
13	ASSURANCE_GAP	Party influence on acquisition is recorded
14	ASSURANCE_GAP	Conflicts are declared rather than concealed
15	OPEN	No differential benefit from a dispute outcome
16	EXTERNAL_QUALIFICATION	The platform does not determine legal liability
17	EXTERNAL_QUALIFICATION	Redaction never modifies the original
18	OPEN	Redacted content is absent from the derivative's bytes
19	ASSURANCE_GAP	VERIQO enforces privilege; it does not determine it
20	ASSURANCE_GAP	View, export, AI processing and training are separate grants

21	ASSURANCE_GAP	A redacted derivative must still pass AI policy
22	EXTERNAL_QUALIFICATION	Every derivative is a new immutable version
23	EXTERNAL_QUALIFICATION	Process evidence is itself evidence
24	ASSURANCE_GAP	Every disclosure emits a ledger event
25	ASSURANCE_GAP	Privilege transitions are immutable events
26	ASSURANCE_GAP	Policy change is never quietly applied to history
27	ASSURANCE_GAP	Material AI contribution is recorded and human-reviewed
28	ASSURANCE_GAP	UNKNOWN independence is never treated as INDEPENDENT
29	ASSURANCE_GAP	OBSERVED_ABSENT only after the observability gate
30	ASSURANCE_GAP	Integrity, provenance, qualification, neutrality and legal dete...

30 articles: 3 OPEN, 0 INTEGRATION_GAP, 18 ASSURANCE_GAP, 9 EXTERNAL_QUALIFICATION, 0 QUALIFIED. QUALIFIED requires an outside party to ha		

CAPABILITY		ENGINEERING ASSURANCE

Evidence Constitution (30 executable articles)	ADVERSARIAL_TESTED	INTERNALLY_PROVED
Epistemic Qualification Fabric	ADVERSARIAL_TESTED	INTERNALLY_PROVED
Proof Object and pipeline	ADVERSARIAL_TESTED	INTERNALLY_PROVED
Case Resolution Fabric	ADVERSARIAL_TESTED	INTERNALLY_PROVED
Forward-Reverse Execution Fabric	UNIT_TESTED	INTERNALLY_PROVED
Trust and Evidence Control Plane	REPLAY_VERIFIED	INTERNALLY_PROVED
Disclosure two-dimensional model	ADVERSARIAL_TESTED	INTERNALLY_PROVED
AI Evidence Gateway	ADVERSARIAL_TESTED	INTERNALLY_PROVED
Temporal attestation distinction	UNIT_TESTED	SELF_ASSERTED
Replay and independent verification	REPLAY_VERIFIED	INTERNALLY_PROVED
Redaction assurance (byte-level absence)	DESIGNED	UNPROVED
Zero-knowledge proofs	DESIGNED	UNPROVED
External source acquisition (IEAP)	DESIGNED	UNPROVED
Payment settlement	IMPLEMENTED	SELF_ASSERTED
Operational neutrality (Article 15)	DESIGNED	UNPROVED

8 of 15 capabilities are adversarially tested or better on the engineering axis.		
0 of 15 have been examined by anyone outside VERIQO.		
These are different axes and neither substitutes for the other.		